Salesforce ↔ ERP 연동 원리

한결공조 AS 관리 시스템에서 Salesforce와 ERP(Mock)가 주고받는 4가지 흐름을, 실제 코드가 하는 일 순서대로 따라가며 정리한 학습 자료

기준: 저장소 `force-app` · `erp-mock/src/index.js` (2026-10-19) · 인터페이스 번호는 docs/interface_definition.md(IF-02·03·06)와 같음

1. 큰 그림 — 누가 원본이고 무엇이 오가나
2. 인증 — 양쪽이 서로를 어떻게 믿나
3. Salesforce → ERP ① 사용 부품 차감 (IF-03)
4. Salesforce → ERP ② 재고 이동 (IF-06)
5. 실패·재전송·중복 방지
6. ERP → Salesforce ① 푸시 (IF-02-P)
7. ERP → Salesforce ② 가져오기 (IF-02-R)
8. 무한 반복을 막는 장치
9. 재적재(seed)와 시연 리셋
10. 로그 · 기록은 어디에 남나
11. 한 장 요약 · 스스로 확인할 질문

1. 큰 그림 — 누가 원본이고 무엇이 오가나

재고의 원본(마스터)은 ERP이고, Salesforce의 재고(`ProductItem`)는 현장에서 쓰기 위한 **사본**입니다. 그래서 두 시스템은 "상대에게 내 쪽에서 생긴 **변동**을 알려 주는" 방식으로 맞춥니다. 수량 자체를 덮어쓰지 않고 **변동량(+/−)만** 주고받는 것이 핵심 원칙입니다.

Salesforce (현장)
기사가 출동에서 **사용 부품** 저장
기사가 **재고 이동**(창고→차량) 수령
ERP에서 온 입고·조정을 재고에 반영
저장소: ProductItem · ProductConsumed · ProductTransfer · ErpInboundLog__c

→ IF-03 차감
→ IF-06 이동
← IF-02-P 푸시
⇠ IF-02-R 가져오기

ERP Mock (원본)
Cloudflare Workers + D1(SQLite)
입고·조정은 ERP 화면에서만 발생
모든 변동을 `movements` 이력으로 번호 매김
테이블: locations · stock · movements · requests · settings

흐름	방향 · 방식	언제	보내는 것
IF-03 사용 부품 차감	SF → ERP, REST POST (Queueable)	사용 부품 저장 직후(비동기)	위치 코드·부품 코드·수량·출동 번호·레코드 Id
IF-06 재고 이동	SF → ERP, REST POST (Queueable)	재고 이동이 "수령"으로 바뀐 직후	출발·도착 위치 코드·부품 코드·수량·이동 번호·Id
IF-02-P 푸시	ERP → SF, Apex REST POST	ERP 화면에서 입고·조정 직후	이력 번호·종류·위치·부품·변동량·단위
IF-02-R 가져오기	SF가 ERP에 GET (Batch)	매시 30분 (푸시를 놓친 것의 안전망)	(요청) 마지막으로 가져온 이력 번호 → (응답) 그 뒤 변동 목록

> **왜 변동량만?** 두 시스템이 동시에 바뀌는 중에 "지금 수량 = 14"처럼 덮어쓰면 사이에 일어난 변동이 사라집니다. "−1", "+2"처럼 변동만 보내고 받는 쪽이 "현재 + 변동"으로 계산하면 순서가 섞여도 결과가 맞습니다. ERP가 돌려주는 남은 수량은 **기록만** 하고 Salesforce 수량을 고치지 않습니다(다르면 불일치 표시).

2. 인증 — 양쪽이 서로를 어떻게 믿나

Salesforce → ERP: Named Credential + API 키 헤더

Apex 코드는 주소나 키를 직접 쓰지 않고 `callout:HG_ErpMock/api/stock/consume`처럼 **Named Credential 이름**으로 호출합니다. 플랫폼이 실제 주소(`https://hg-erp-mock.hg-erp-mock.workers.dev`)를 붙이고, External Credential `HG_ErpMock`의 보안 주체 `HG_ErpPrincipal`에 저장된 키를 헤더에 넣어 줍니다.

```
Named Credential HG_ErpMock
  URL            https://hg-erp-mock.hg-erp-mock.workers.dev
  사용자 지정 헤더  x-api-key = {!$Credential.HG_ErpMock.ApiKey}   ← 실행 시점에 키로 치환
ERP Worker
  if (!env.API_KEY || request.headers.get("x-api-key") !== env.API_KEY) → 401
```

- 키는 메타데이터·Git·로그에 없습니다(설정 화면에서 입력, ERP 쪽은 Cloudflare secret `API_KEY`).
- 실행 사용자에게 **보안 주체 접근 + UserExternalCredential 읽기**가 둘 다 있어야 키가 채워집니다. 기사(최민호)가 저장한 사용 부품을 기사 권한으로 전송하기 때문에 기사 권한 집합에도 들어 있습니다. HttpCalloutMock 테스트로는 이 부분을 잡을 수 없어 실제 사용자로 확인했습니다.

ERP → Salesforce: OAuth 2.0 Client Credentials

사람이 없는 서버 간 호출이라 로그인 화면 없이 **클라이언트 ID·비밀값으로 토큰**을 받습니다. Salesforce의 외부 클라이언트 앱 "HG ERP Mock"이 토큰을 내주고, 그 토큰은 **Run As 사용자 "ERP 연동"**의 권한으로 동작합니다.

```
① POST {SF_INSTANCE_URL}/services/oauth2/token
     content-type: application/x-www-form-urlencoded
     grant_type=client_credentials&client_id=…&client_secret=…
   ← { "access_token": "…", "instance_url": "https://…my.salesforce.com" }
② POST {instance_url}/services/apexrest/hg/erp/movements
     authorization: Bearer {access_token}
   - 토큰은 Worker 메모리에 10분 재사용, 401이 오면 한 번만 새 토큰으로 다시 보냄
```

- 연동 사용자 권한은 최소: Apex 클래스 실행(`HG_ErpIntegration`) + 재고·위치·제품 읽기(`HG_ErpIntegrationData`, Field Service 플랫폼 검사가 요구). 실제 저장은 서비스 클래스가 시스템 모드로 합니다.
- 클라이언트 ID·비밀값은 Cloudflare secret `SF_CLIENT_ID`·`SF_CLIENT_SECRET`. 설정돼 있는지는 `/health`의 `push: configured`로 확인합니다.

3. Salesforce → ERP ① 사용 부품 차감 (IF-03)

기사가 출동 화면에서 부품을 추가해 `ProductConsumed`가 저장되면 일어나는 일을 순서대로 봅니다.

1	**트리거**	`ProductConsumedTrigger` → `ProductConsumedTriggerHandler`. **before insert**: 차량 재고 초과 거부(VanStockService), 유·무상 판정(WarrantyService). **after insert**: 저장된 Id 묶음으로 `ErpInventoryService.enqueueConsumed(ids)`
2	**예약**	`System.enqueueJob(new ErpSyncQueueable(ids, null, false))` — 트리거 안에서는 외부 호출(콜아웃)을 할 수 없어서 저장이 끝난 뒤 도는 **비동기 작업**에 넘깁니다. 같은 저장에서 Salesforce 표준 기능이 차량 재고를 이미 차감합니다.
3	**Queueable**	`ErpSyncQueueable.execute()` → `buildConsumedUpdates(ids, isRetry)`: 사용 부품과 위치 코드(`Location.ErpLocationCode__c`)·부품 코드(`Product2.ErpPartCode__c`)를 조회해 보낼 목록을 만듦. 코드가 없으면 보내지 않고 바로 "확인 필요 [MISSING_CODE]"
4	**HTTP 전송**	`post()`가 **200건씩** 잘라 `postChunk()`로 한 번에 보냄(200건 = 콜아웃 1번). `HttpRequest`: POST · 시간 제한 60초 · JSON 본문
5	**ERP 처리**	`consume()`: 건마다 ① 같은 refId 처리 기록(`requests`)이 있으면 그 결과를 그대로 반환 ② 수량·위치·부품 검사 ③ `decrease()`로 재고 차감 + `movements`에 CONSUME 이력 ④ 결과를 `requests`에 저장
6	**결과 해석**	`interpret()`로 건별 상태 결정 → 완료면 ERP 남은 수량 기록, Salesforce 재고와 다르면 불일치 체크
7	**결과 저장**	`save()`: 사용 부품의 ERP 필드만 시스템 모드로 update. **콜아웃을 모두 끝낸 뒤 한꺼번에 저장**(저장 뒤에는 같은 트랜잭션에서 콜아웃 불가)

실제로 오가는 데이터

```
POST callout:HG_ErpMock/api/stock/consume        (헤더 x-api-key, content-type: application/json)
{ "items": [
    { "refId": "0Q7aj0000…", "locationCode": "VAN-GS-001", "partCode": "HG-P-FM30",
      "quantity": 1, "workOrderNumber": "00000051" } ] }

← 200
{ "results": [
    { "refId": "0Q7aj0000…", "status": "OK", "remainingQty": 5, "unit": "Each" } ] }
  거부 예: { "refId": "…", "status": "REJECTED", "code": "INSUFFICIENT_STOCK",
           "message": "재고가 부족합니다. 현재 0개, 요청 1개", "availableQty": 0 }
```

ERP는 **업무 결과(성공·거부)는 HTTP 200 + 건별 status**로, 요청 자체의 문제는 HTTP 코드(인증 401, 형식 400, 장애 503)로 구분합니다. 그래서 200건 중 일부만 거부돼도 나머지는 정상 처리됩니다.

ERP가 재고를 줄이는 SQL — 음수가 안 되는 이유

```
UPDATE stock SET quantity = ROUND(quantity - ?, 2), updated_at = ?
 WHERE location_code = ? AND part_code = ? AND quantity >= ?      ← 조건에 "남은 수량 ≥ 요청"
-- 바뀐 행이 1이 아니면(조건 불충족) → INSUFFICIENT_STOCK
INSERT INTO movements (ref_id, type, …, quantity_change, quantity_after, origin) VALUES (…, 'CONSUME', …,
-1, 5, 'SALESFORCE')
```

> "조회해서 확인 → 빼기"를 두 번에 나누면 동시에 두 요청이 오면 둘 다 통과할 수 있습니다. **조건부 UPDATE 한 문장**으로 확인과 차감을 한 번에 하므로 동시 요청에도 음수가 되지 않습니다.

사용 부품에 남는 필드

필드	뜻
`ErpSyncStatus__c`	Pending(대기) → Synced(완료) / Failed(실패, 재전송 대상) / NeedsReview(확인 필요)
`ErpSyncMessage__c`	"[코드] 내용" 형식, 255자. 예: `[OK] ERP 재고 차감 완료`, `[INSUFFICIENT_STOCK] 재고가 부족합니다…`, `[HTTP_503] …`
`ErpSyncedAt__c` · `ErpRetryCount__c`	마지막 시도 시각 · 재전송 횟수
`ErpRemainingQty__c` · `InventoryMismatch__c`	ERP가 알려 준 남은 수량 · Salesforce 수량과 다르면 체크(수량은 안 고침)

4. Salesforce → ERP ② 재고 이동 (IF-06)

창고 → 기사 차량 보충(`ProductTransfer`)은 **"수령"되는 순간**에만 보냅니다. 만들기만 하고 받지 않은 이동은 실제로 재고가 움직이지 않았기 때문입니다.

1	**before**	`ProductTransferTriggerHandler` before insert·update: 이번 저장에서 `IsReceived`가 false → true가 된 건(또는 수령 상태로 생성)을 **Pending**으로 표시
2	**after**	수령된 건만 `enqueueTransfers(ids)` → 같은 `ErpSyncQueueable`
3	**전송**	`buildTransferUpdates()` → `POST callout:HG_ErpMock/api/stock/transfer`. 수량 = 받은 수량(없으면 보낸 수량), 참조 = 재고 이동 번호
4	**ERP**	`transfer()`: 출발지 `decrease()`(TRANSFER_OUT) → 도착지 `increase()`(TRANSFER_IN). 도착지에 그 부품 재고가 없으면 `INSERT … ON CONFLICT DO UPDATE`로 새로 만듦
5	**결과**	출발·도착 남은 수량(`ErpSourceRemainingQty__c`·`ErpDestinationRemainingQty__c`) 기록, 둘 중 하나라도 Salesforce와 다르면 불일치

```
{ "items": [ { "refId": "0Luaj…", "fromLocationCode": "WH-GS-001", "toLocationCode": "VAN-GS-001",
               "partCode": "HG-P-DP4", "quantity": 2, "reference": "PT-0012" } ] }
← { "results": [ { "refId": "0Luaj…", "status": "OK", "sourceRemainingQty": 17,
                   "destinationRemainingQty": 3, "unit": "Each" } ] }
```

> **같은 실행에서 이동과 사용을 함께 보내면?** 둘 다 비동기 작업이라 ERP에 도착하는 순서가 바뀔 수 있습니다. 2026-10-18 시연 준비 때 "차량 보충 → 압축기 사용"을 한 스크립트에서 저장했더니 사용 차감이 먼저 도착해 "재고 부족(현재 0개)"이 났습니다(PC-269). 화면에서는 이동 수령과 부품 사용이 따로 저장되어 생기지 않습니다.

5. 실패·재전송·중복 방지

결과 → 상태 규칙

상황	상태	이유
ERP가 건별 `status: "OK"`	Synced	완료
ERP가 건별 거부(재고 부족·없는 위치/부품 코드·잘못된 수량)	NeedsReview	다시 보내도 같은 결과 → 사람이 확인
HTTP 400(요청 형식)	NeedsReview	다시 보내도 같은 결과
HTTP 401·5xx, 시간 초과·연결 실패 `[CONNECTION]`, 응답 해석 불가 `[BAD_RESPONSE]`	Failed	일시적 → 재전송 대상
재전송에서도 실패해 횟수가 상한(ServiceSetting `ErpMaxRetry` = 3)에 도달	NeedsReview	`[RETRY_EXHAUSTED] 재시도 3회 초과. 마지막 오류: …`

재전송 Batch

예약 작업 "HG ERP 재전송"(매시 정각) → `ErpRetrySchedulable` → `ErpRetryBatch`. 대상은 ① 실패이면서 재시도 횟수 < 상한 ② 10분 넘게 대기로 남은 건(Queueable 한도 초과 등으로 처음부터 못 간 것)입니다. 같은 `buildConsumedUpdates/buildTransferUpdates`를 `isRetry = true`로 불러 실패 시 횟수를 1 올립니다. 관리자 홈의 "ERP 실패·확인 필요" 목록이 이 두 상태를 보여 줍니다.

중복 방지(멱등성) — 두 번 보내도 한 번만 빠지게

Salesforce는 매 건에 **레코드 Id(refId)**를 붙여 보냅니다. ERP는 처음 처리한 결과를 `requests(ref_id PK, result_json)`에 저장하고, 같은 refId가 다시 오면 **재고를 건드리지 않고 저장된 결과를 그대로** 돌려줍니다. 그래서 응답이 중간에 끊겨 Salesforce가 "실패"로 알고 다시 보내도 이중 차감이 생기지 않습니다.

> **멱등성의 뒷면:** 처음 결과가 "재고 부족"이면 나중에 재고가 채워져도 같은 refId로는 영원히 "재고 부족"이 돌아옵니다. PC-269는 다시 보내도 같은 응답이 와서 **사용 부품을 지우고 새로 넣어(새 Id)** 해결했습니다. 확인 필요(NeedsReview) 건은 이렇게 "지우고 다시 입력"이 정상 처리 방법이고, 그래서 삭제 규칙이 실패·확인 필요 건만 삭제를 허용합니다(대기·완료는 거부).

사용 부품을 지우면 Salesforce 차량 재고는 표준 기능이 되돌리지만 ERP에는 "되돌림"을 보내지 않습니다. 확인 필요 건은 ERP에서 애초에 차감되지 않았으므로 맞고, 완료 건은 삭제 자체를 막습니다.

6. ERP → Salesforce ① 푸시 (IF-02-P)

ERP 화면에서 입고(RECEIPT)나 조정(ADJUST)을 입력하면 ERP 재고가 바뀌고, **그 즉시** Salesforce에 알려 줍니다.

1	**입고 저장**	`receive()` / `adjust()` → `increase()` / `decrease()` (origin = **ERP**) → `movements`에 새 이력 번호 (예: #54)
2	**푸시**	`pushMovement()` : 토큰을 받아(2절) Apex REST로 1건 POST. 8초 안에 응답이 없으면 실패 처리
3	**입구**	`ErpMovementResource.receive()` (`@RestResource(urlMapping='/hg/erp/movements')`, `@HttpPost`): 본문 크기(1MB)·건수(200)·종류·코드 형식·수량 범위·번호 검사. 요청 전체가 잘못되면 400, 건별 오류는 INVALID
4	**반영**	`ErpStockSyncService.applyPushed()` : Savepoint → ERP 출처 RECEIPT·ADJUST만 → 이미 기록된 이력 번호면 DUPLICATE → 위치·부품을 코드로 찾음 → 재고를 `FOR UPDATE`로 잠그고 **현재 + 변동량** → 재고가 없으면 새로 만듦(소유자 = 위치 소유자) → `ErpInboundLog__c` 기록
5	**화면 표시**	응답의 결과를 화면에 "Salesforce: 즉시 반영됨"으로 보여 줌. 실패해도 ERP 저장은 그대로이고 **ERP는 재시도하지 않음** → 7절 가져오기가 다시 가져감

```
POST /services/apexrest/hg/erp/movements      authorization: Bearer …
{ "movements": [ { "id": 54, "type": "RECEIPT", "origin": "ERP", "locationCode": "WH-GS-001",
                   "partCode": "RFG-404A", "quantityChange": 1, "unit": "kg",
                   "reference": "ERP 입고", "createdAt": "2026-10-16T08:46:33Z" } ] }
← 200 { "results": [ { "id": 54, "result": "APPLIED" } ], "applied": 1, "duplicates": 0, "skipped": 0,
"errors": 0 }
```

> **왜 `without sharing`?** 연동 사용자는 레코드를 하나도 소유하지 않고 위치·재고 공유도 없습니다. 조회를 시스템 모드로 해도 **레코드 공유는 클래스의 sharing 선언**을 따르기 때문에, 처음 실제 호출(2026-10-15)에서 "없는 위치 코드" 오류가 났습니다. 입구 클래스를 without sharing으로 두고, 대신 **클래스 실행 권한 + 입력 검증**을 관문으로 삼았습니다.

7. ERP → Salesforce ② 가져오기 (IF-02-R, 안전망)

푸시가 네트워크 문제 등으로 빠질 수 있으므로, Salesforce가 매시 30분에 ERP 이력을 직접 조회해 빠진 것을 채웁니다.

1	**예약**	"HG ERP 재고 동기화 :30 (매시, 푸시 안전망)" → `ErpStockSyncSchedulable` → `ErpStockSyncBatch` (묶음 1개 = 이력 한 쪽)
2	**기준 번호**	Custom Setting `ErpSyncState__c.LastMovementId__c` = 마지막으로 가져온 이력 번호(예: 89)
3	**조회**	`GET callout:HG_ErpMock/api/movements?after=89&limit=200`
4	**ERP**	`getMovements()` : `SELECT … FROM movements WHERE id > ? ORDER BY id LIMIT 201` — 한 건 더 읽어 다음 쪽 여부(`hasMore`) 판단
5	**반영**	푸시와 **같은 반영 규칙**(ERP 출처 입고·조정만, 중복 건너뜀) + 이번 쪽의 마지막 번호를 `LastMovementId__c` 에 저장 — **반영·기록·번호 저장을 한 트랜잭션**으로

```
← { "items": [ { "id": 90, "type": "CONSUME", "origin": "SALESFORCE", … },      ← 건너뜀(이미 SF에 있음)
              { "id": 91, "type": "ADJUST",  "origin": "ERP", "quantityChange": -1, … } ],  ← 반영
    "lastId": 91, "hasMore": false, "latestId": 91 }
```

실패 종류	처리
통신 실패(연결·HTTP 오류)	예외 → Savepoint로 되돌림 → **기준 번호를 올리지 않음** → 다음 실행에서 같은 쪽부터 다시
데이터 문제(없는 위치·부품 코드, 음수가 되는 조정)	다시 해도 같은 결과 → `ErpInboundLog__c` 에 Error로 기록하고 건너뜀(번호는 진행). 재시도하면 영원히 막히기 때문

> **푸시와 가져오기가 같은 변동을 받으면?** 둘 다 반영 전에 `ErpInboundLog__c.MovementId__c` (이력 번호, **고유**)를 확인해 이미 있으면 건너뜁니다. 확인과 저장 사이에 동시에 들어와도 고유 제약 때문에 두 번째 저장이 거부되고 트랜잭션 전체가 되돌아가 이중 반영이 없습니다. 기준 번호는 가져오기만 올립니다.

8. 무한 반복을 막는 장치

- **ERP는 Salesforce 출처 변동을 다시 푸시하지 않습니다.** 푸시는 `receive` / `adjust` (origin ERP)에서만 호출됩니다.
- **Salesforce는 받은 변동을 재고 수량 조정으로만 반영합니다.** 사용 부품·재고 이동 레코드를 만들지 않으므로 ERP 전송 트리거가 다시 돌지 않습니다.
- 가져오기도 origin SALESFORCE·SEED 이력은 건너뜁니다(이미 Salesforce에서 생긴 변동).

9. 재적재(seed)와 시연 리셋

`scripts/apex/d7_erp_seed.apex` 는 Salesforce의 현재 창고·차량 재고를 `POST /api/admin/seed {reset: true, locations, stock}` 로 보내 ERP를 그 값으로 다시 채웁니다. ERP의 `stock` · `movements` · `requests` 를 비우고 SEED 이력을 남기며, **이력 번호는 자동 증가라 이어서 올라가** 가져오기 기준 번호와 어긋나지 않습니다. 아직 ERP에 안 간 대기·실패 건이 있으면 재적재 뒤 재전송 때 이중 차감되므로, 그런 건이 있으면 스크립트가 멈춥니다(2026-10-13 사고에서 얻은 규칙). 시연 리셋은 Salesforce를 기준값으로 맞춘 뒤 이 재적재로 ERP를 맞춥니다.

10. 로그 · 기록은 어디에 남나

결론부터: **"통신 원문 로그"를 따로 쌓는 곳은 없습니다.** 대신 업무 결과가 레코드·테이블에 남습니다.

위치	무엇이	얼마나	한계
SF 사용 부품·재고 이동의 ERP 필드	마지막 전송 상태·메시지("[코드] 내용")·시각·재시도 횟수·ERP 남은 수량·불일치	레코드가 있는 동안	**마지막 시도만**(덮어씀). 필드 변경 이력 추적이 꺼져 있어 시도별 이력은 없음. 요청·응답 원문 없음
SF `ErpInboundLog__c` (목록 "전체 기록")	ERP에서 받은 입고·조정 1건당 1레코드: 이력 번호(고유)·종류·위치·부품·변동량·결과(Applied/Error)·메시지(끝에 "· 푸시"/"· 가져오기")	지우지 않는 한 영구	ERP 출처 입고·조정만. SF 출처·SEED·형식 오류(INVALID)는 기록 안 함
SF `ErpSyncState__c`	가져오기 기준 번호 하나	현재 값만	—
SF 설정 → Apex 작업	Queueable·Batch 실행 상태·오류 메시지	플랫폼이 일정 기간만 보관	어떤 레코드를 보냈는지는 안 보임
SF 디버그 로그	코드 실행 상세	추적 플래그를 켠 동안만, 짧게 보관	평소에는 남지 않음
ERP `movements`	모든 재고 변동(누가·어디·얼마·변동 후 수량·참조)	재적재 전까지	재적재(seed) 때 비워짐
ERP `requests`	Salesforce 레코드 Id별 첫 처리 결과(JSON)	재적재 전까지	재적재 때 비워짐
ERP 푸시 결과	입고·조정 화면에 한 번 표시	**저장 안 함**	실패해도 기록 없음(가져오기가 보완)
Cloudflare Worker 로그	—	**설정 안 함**	`wrangler.toml` 에 observability 없음, 코드에 console 로그 없음 → 실시간 `npx wrangler tail` 로만 볼 수 있음

필요하면 다음 단계로 할 수 있는 것
① Salesforce 송신 기록 오브젝트(전송 1회당 요청 건수·HTTP 코드·응답 요약·소요 시간) — 지금은 레코드에 마지막 결과만 남음
② Cloudflare Workers Logs 켜기(`[observability] enabled = true`) + 주요 처리에 `console.log` — 대시보드에서 요청별 기록 조회
③ 사용 부품·재고 이동의 `ErpSyncStatus__c` 필드 변경 이력 추적 — 상태가 바뀐 순서를 볼 수 있음

11. 한 장 요약 · 스스로 확인할 질문

개념	이 프로젝트에서
트리거에서 콜아웃 금지	트리거는 Queueable 예약만, 실제 호출은 `ErpSyncQueueable`
콜아웃 뒤 DML 순서	사용 부품·재고 이동을 모두 보낸 뒤 결과를 한꺼번에 저장
벌크	200건 = 콜아웃 1번(`items` 배열)
오류 분류	일시적(Failed, 재전송) vs 영구적(NeedsReview, 사람 확인)
멱등성	SF→ERP는 refId(`requests`), ERP→SF는 이력 번호(`ErpInboundLog__c.MovementId__c` 고유)
원자성	ERP 조건부 UPDATE, SF Savepoint + FOR UPDATE + 한 트랜잭션
푸시 + 가져오기	빠르게(푸시) + 확실하게(가져오기), 기준 번호는 가져오기만 올림
비밀 관리	Named/External Credential, Cloudflare secret — 코드·Git·로그에 없음

질문

1. 사용 부품을 저장한 같은 트랜잭션에서 바로 ERP를 부르면 어떤 오류가 날까?
2. ERP 응답이 오는 도중 연결이 끊겨 Salesforce가 "실패"로 표시했다. 재전송하면 재고가 두 번 빠질까? 왜?
3. 처음 전송이 "재고 부족"이었는데 지금은 재고가 있다. 재전송 Batch가 해결해 줄까?
4. 푸시가 성공한 변동을 30분 뒤 가져오기가 또 받으면 어떻게 될까?
5. 가져오기 중 ERP가 503을 돌려주면 기준 번호는 어떻게 되나? 없는 부품 코드가 오면?
6. ERP에서 입고하면 Salesforce 재고가 바뀌는데, 그때 ERP로 다시 전송되지 않는 이유는?

답: 1 콜아웃 금지(트리거) · 2 아니오, refId로 처음 결과 반환 · 3 아니오(같은 응답), 지우고 다시 입력 · 4 DUPLICATE로 건너뜀 · 5 그대로(다음에 다시) / Error 기록 후 진행 · 6 재고 수량만 조정하고 사용 부품·재고 이동을 만들지 않음